Intuition behind Attention

🧠 Big Picture: What is Attention?

Attention answers:

“For each word, which other words should I focus on—and how much?”

Instead of processing text sequentially (like RNNs), transformers **look at all words simultaneously** and compute relationships between them.

📌 Step 0: Our Simple Example Sentence

Let’s use a tiny sentence:

“The cat sat”

We’ll track how the word **“sat”** understands context using attention.

🧱 Step 1: Tokenization

We split text into tokens:

["The", "cat", "sat"]

Each token becomes a vector.

🔢 Step 2: Embedding (Turning Words into Numbers)

Each word is mapped to a vector of size d_model (say 4 for simplicity):

Word	Embedding
The	[1, 0, 1, 0]
cat	[0, 1, 1, 0]
sat	[1, 1, 0, 0]

👉 These embeddings are **learned representations** capturing meaning.

📍 Step 3: Positional Encoding (Order Matters!)

Transformers don't know word order naturally.

So we add positional encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

👉 For simplicity, assume:

Position Encoding

0 (The) [0.1, 0.2, 0.1, 0.2]

1 (cat) [0.2, 0.1, 0.2, 0.1]

2 (sat) [0.3, 0.3, 0.1, 0.1]

✅ Final Input Representation

We **add embedding + positional encoding**:

The = [1.1, 0.2, 1.1, 0.2]
cat = [0.2, 1.1, 1.2, 0.1]
sat = [1.3, 1.3, 0.1, 0.1]

🔁 Step 4: Creating Q, K, V (Core of Attention)

Each word is transformed into:

- **Query (Q)** → what this word is looking for
- **Key (K)** → what this word offers
- **Value (V)** → actual information

We compute:

$$Q = XW_Q, K = XW_K, V = XW_V$$

Assume simple weight matrices:

W_Q, W_K, W_V → (4×4 matrices)

Example (for "sat")

After multiplication:

Q_sat = [1, 0, 1, 0]
K_sat = [0, 1, 0, 1]
V_sat = [1, 1, 0, 0]

Similarly for other words.

⚡ Step 5: Attention Score Calculation

We compute how much "sat" attends to each word:

$$\text{score} = Q \cdot K^T$$

👉 Dot product = similarity

Example: "sat" attending

Pair	Score
sat → The	1.2
sat → cat	2.1
sat → sat	1.5

👉 Interpretation:

- "sat" relates most to **"cat"**

🔥 Step 6: Scaling

To stabilize gradients:

$$\text{scaled score} = \frac{QK^T}{\sqrt{d_k}}$$

If $d_k = 4$, divide by 2.

🎯 Step 7: Softmax (Convert to Probabilities)

$$\text{Attention Weights} = \text{softmax}(\text{scores})$$

Example:

[1.2, 2.1, 1.5] → softmax → [0.2, 0.5, 0.3]

👉 Meaning:

- 20% attention → "The"
- 50% attention → "cat"
- 30% attention → "sat"

📦 Step 8: Weighted Sum of Values

$$\text{Output} = \sum(\text{attention weight} \times V)$$

Output_sat =
0.2 * V_The +
0.5 * V_cat +
0.3 * V_sat

👉 This creates a **context-aware representation of "sat"**

🧠 Interpretation

Originally:

sat = [1.3, 1.3, 0.1, 0.1]

After attention:

sat = context-aware vector

👉 Now it "knows":

- Who is sitting? → "cat"
- Context of action

🔁 Step 9: Matrix Form (Efficient Computation)

Instead of doing this per word:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This is the **core transformer equation**.

🎯 Key Insight

Attention =

Compare (Q vs K) → Decide importance → Combine (V)

🧩 Why Q, K, V?

Think of it like:

Concept	Meaning
Query	“What am I looking for?”
Key	“What do I contain?”
Value	“What information I provide”

🚀 What This Achieves

For **“sat”**, attention helps:

- Link to subject → "cat"
- Ignore less relevant words → "The"
- Build semantic understanding

🔥 Final Intuition

Instead of:

❌ Fixed representation of a word
✅ Context-aware representation

📌 One-Line Summary

Attention lets each word dynamically gather information from all other words based on relevance.

🧠 Big Picture (Reframed)

Attention is not magic—it is just:

learned linear projections + similarity + weighted averaging

The **most important idea**:
👉 Q, K, V are *not arbitrary*—they are **learned transformations of embeddings**.

📌 Step 0: Example Sentence

We use:

"The cat sat"

Tokens:

["The", "cat", "sat"]

🔢 Step 1: Embeddings

Assume embedding dimension $d_{model} = 4$

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

Each row = word vector.

📍 Step 2: Positional Encoding

We add positional encoding:

$$X' = X + PE$$

Assume:

$$X' = \begin{bmatrix} 1.1 & 0.2 & 1.1 & 0.2 \\ 0.2 & 1.1 & 1.2 & 0.1 \\ 1.3 & 1.3 & 0.1 & 0.1 \end{bmatrix}$$

🔥 Step 3: Why Do We Need Q, K, V?

Before jumping into formulas, understand this:

👉 A **single embedding vector cannot simultaneously**:

- Ask a question (Query)
- Represent content (Value)
- Be searchable (Key)

So we **split roles**:

Role	Purpose
Query (Q)	What am I looking for?
Key (K)	What do I contain?
Value (V)	What information do I give?

🚀 Step 4: Deriving Q, K, V (DETAILED CORE)

This is the heart of transformers.

🔹 4.1 Linear Transformation

We compute:

$$Q = X'W_Q, K = X'W_K, V = X'W_V$$

Where:

- X': input matrix (3 × 4)
- W_Q, W_K, W_V: learned weight matrices (4 × 4)

🔹 4.2 What Are These Weight Matrices?

They are just **learned parameters**, like in any neural network.

Example:

$$W_Q = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$
$$W_K = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$
$$W_V = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

👉 These are initialized randomly and **learned during training via backpropagation**.

🔹 4.3 Compute Q for Each Word (Step-by-Step)

Take **"sat"**:

$$X'_{sat} = [1.3, 1.3, 0.1, 0.1]$$

Now:

$$Q_{sat} = X'_{sat} \cdot W_Q$$

Multiply row × matrix:

$$Q_{sat}[0] = 1.3(1) + 1.3(0) + 0.1(1) + 0.1(0) = 1.4$$
$$Q_{sat}[1] = 1.3(0) + 1.3(1) + 0.1(0) + 0.1(1) = 1.4$$
$$Q_{sat}[2] = 1.3(1) + 1.3(0) + 0.1(0) + 0.1(1) = 1.4$$
$$Q_{sat}[3] = 1.3(0) + 1.3(1) + 0.1(1) + 0.1(0) = 1.4$$

✅ **Result:**

Q_sat = [1.4, 1.4, 1.4, 1.4]

🔹 4.4 Compute K and V Similarly

You repeat the same multiplication:

$$K_{sat} = X'_{sat} \cdot W_K$$
$$V_{sat} = X'_{sat} \cdot W_V$$

👉 Each gives a **different projection of the same word**.

🧠 KEY INSIGHT (VERY IMPORTANT)

Q, K, V are **not new information**—they are **different views of the same embedding**

Think of it like:

- Q = "question vector"
- K = "index vector"
- V = "content vector"

🎯 Why Linear Transformation Works

Matrix multiplication does:

$$\text{new feature} = \sum(\text{old features} \times \text{weights})$$

👉 It allows the model to learn:

- syntactic roles (subject/object)
- semantic relationships (who did what)

⚡ Step 5: Attention Scores

Now compute:

$$\text{score} = QK^T$$

For "sat":

$$Q_{sat} \cdot K_{cat}$$

👉 Dot product = similarity

📉 Step 6: Scaling

$$\frac{QK^T}{\sqrt{d_k}}$$

Prevents large values.

🎯 Step 7: Softmax

$$\text{softmax}(scores)$$

Converts to probabilities.

📦 Step 8: Weighted Sum

$$\text{Output} = \text{softmax}(QK^T)V$$

🔁 Final Equation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

🧠 Deep Intuition for Step 4 (Final Takeaway)

👉 The **entire power of attention comes from these matrices**:

$$W_Q,\ W_K,\ W_V$$

They learn:

- what relationships matter
- how to compare words
- what information to extract

🔥 Analogy (Makes It Click)

Imagine a library:

- Query = your search question
- Key = book titles
- Value = book content

👉 Attention =
match question to titles → retrieve relevant content

🚀 What You Should Now Clearly Understand

After this:

✅ Q, K, V are **learned projections**
✅ They come from **matrix multiplication**
✅ They enable **comparison + information flow**
✅ Attention = **similarity + weighted aggregation**

FAQs

🔤 Step 1: A tiny sentence (6 tokens)

We use:

"the cat sat on the mat"

So:

- Number of tokens = **6**

🔢 Step 2: Convert tokens → embeddings

Each token becomes a vector of size **d_model = 4** (chosen small for clarity)

$$X \in \mathbb{R}^{6\times 4}$$

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

❓ Why 6 × 4?

- 6 = number of tokens
- 4 = embedding size (you choose this; real models use 512, 768, etc.)

🧠 Step 3: Learn projection matrices

We create 3 matrices:

$$W_Q, W_K, W_V \in \mathbb{R}^{4\times 3}$$

❓ Why 4 × 3?

- Input vectors are size **4**
- We want Q, K, V to be size **3**
- So matrix must map:

$$4 \rightarrow 3$$

👉 That means:

$$(6 \times 4) \cdot (4 \times 3) = (6 \times 3)$$

🔁 **Step 4: Compute Q, K, V**

$$Q = XW_Q, K = XW_K, V = XW_V$$

So:

$$Q, K, V \in \mathbb{R}^{6\times 3}$$

❓ **Why 6 × 3?**

- 6 rows → one per token
- 3 columns → new feature space (called **d_k or d_v**)

👉 Each token now has:

- a **Query vector (3D)**
- a **Key vector (3D)**
- a **Value vector (3D)**

🔍 **Step 5: Compute attention scores**

$$\text{Scores} = QK^T$$

Shapes:

- $Q = (6 \times 3)$
- $K^T = (3 \times 6)$

$$\Rightarrow (6 \times 6)$$

❓ Why 6 × 6?

Because:

- Each of 6 tokens compares with **all 6 tokens**

👉 Entry (i, j) =
"How much should token *i* pay attention to token *j*?"

🎯 Step 6: Scale + Softmax

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{3}}\right)$$

❓ Why divide by √3?

- 3 = dimension of K (d_k)
- Prevents values from becoming too large
- Keeps softmax stable

📦 Step 7: Multiply with V

$$\text{Output} = \text{Attention} \cdot V$$

Shapes:

- (6 × 6) × (6 × 3) → **(6 × 3)**

❓ Why this works?

Each row of Attention = weights
Each row sums to 1

👉 So output = weighted sum of all value vectors

🧠 Full Flow (Shapes Only)

$$X(6 \times 4) \rightarrow Q, K, V(6 \times 3) \rightarrow QK^T(6 \times 6) \rightarrow \text{softmax}(6 \times 6) \rightarrow \text{Output}(6 \times 3)$$

💡 Intuition (Super Important)

For each token:

- **Query (Q)** → What am I looking for?
- **Key (K)** → What do I offer?
- **Value (V)** → What information do I give?

🧩 Example: token "cat"

- Q(cat) compares with:
  - K(the), K(sat), K(on), ...
- Suppose:
  - high similarity with "sat"
- Then:
  - output(cat) = mostly V(sat)

👉 So "cat" learns from "sat"

🧠 Why not just use X directly?

Because:

- Q, K, V allow **different roles**
- Same token can:
  - ask different questions (Q)
  - describe itself differently (K)
  - pass different info (V)

❓ FAQs (Beginner-Friendly, Deep Curiosity)

---

1. Why do we need Q, K, V at all?

To separate:

- matching (Q vs K)
- information passing (V)

---

2. Why three different matrices?

Because:

- Matching and information are different tasks

---

3. Can Q = K = V?

Yes, but performance drops — model becomes less expressive

---

4. Why is X multiplied three times?

To create three different perspectives of the same data

---

5. Why dot product for attention?

- Fast
- Works well for similarity

---

6. Why not cosine similarity?

Dot product is simpler and works similarly with scaling

---

7. Why softmax?

To convert scores into probabilities (sum = 1)

---

8. Why rows sum to 1?

So each token distributes its attention across others

---

9. Why does each token attend to itself?

Self-attention includes self-information

---

10. Why dimension 3? Can it be anything?

Yes:

- Typically 64, 128 in real models
- We used 3 for simplicity

---

11. Why reduce from 4 → 3?

To:

- reduce compute
- create compact representations

---

12. What happens if dimension is too small?

Model may lose information

---

13. What happens if too large?

- Slower
- Overfitting risk

---

14. Why multiply with K^T?

To compare every query with every key

---

15. Why output is again per token?

Each token gets a **context-aware representation**

16. Is attention symmetric?

No:

- attention(i → j) ≠ attention(j → i)

---

17. What is multi-head attention?

Repeat this process multiple times with different weights

---

18. Why multiple heads?

Each head learns different relationships:

- syntax
- semantics
- position

---

19. Does attention capture word order?

Not by itself → needs positional encoding

---

20. Is this used only in NLP?

No:

- vision (ViT)
- audio
- multimodal models

⚡ **Final Mental Model**

👉 Each token:

1. asks a question (Q)
2. compares with all tokens (K)
3. gathers information (V)

👉 Result: smarter, context-aware tokens